



ROYAUME DU MAROC

Université Abdelmalek Essaâdi

École Nationale des Sciences Appliquées d'Al Hoceima

Rapport de TP N° 5

La Généricité & La Gestion des Exceptions

Programmation Orientée Objet – Java SE

Réalisé par :

Yazid TAHIRI ALAOUI

Filière : 1ère année – Ingénierie Données (ID1)

Encadré par : Pr. Mohamed CHERRADI

Année universitaire : 2025 – 2026

Résumé

Ce rapport présente les travaux réalisés dans le cadre du TP N°5 du module **Programmation Orientée Objet en Java SE**, portant sur deux concepts fondamentaux du langage Java : la **gestion des exceptions** et la **généricité**.

Le TP est organisé en trois parties : la première traite des exceptions système (vérifiées et non vérifiées) à travers des exemples concrets de division, de saisie utilisateur, de tableaux et de propagation d'exceptions.

La deuxième partie porte sur la création d'exceptions personnalisées appliquées à des cas métier tels que les comptes bancaires, la gestion de stock et les formulaires. La troisième partie explore la généricité avec des classes, interfaces, méthodes génériques, bornes et wildcards.

Mots-clés : Java, Exceptions, Généricité, Checked, Unchecked, Wildcards, Programmation Orientée Objet.

Table des matières

Résumé	i
1 Exceptions Système	1
1.1 Exercice 1 – Division Sécurisée	1
1.1.1 Objectif	1
1.1.2 Code source	1
1.1.3 Explication	2
1.2 Exercice 2 – NullPointerException	2
1.2.1 Objectif	2
1.2.2 Code source	2
1.2.3 Explication	3
1.3 Exercice 3 – Tableau Sécurisé	3
1.3.1 Objectif	3
1.3.2 Code source	3
1.3.3 Comparaison des deux approches	4
1.4 Exercice 4 – Conversion d'Entier	4
1.4.1 Objectif	4
1.4.2 Code source	4
1.4.3 Explication	4
1.5 Exercice 5 – Choix d'Exception	4
1.5.1 Objectif	4
1.5.2 Code source	5
1.5.3 Justification du choix	5
1.6 Exercice 6 – IllegalStateException	5
1.6.1 Objectif	5
1.6.2 Code source	5
1.6.3 Explication	6
1.7 Exercice 7 – Propagation d'Exception	6
1.7.1 Objectif	6
1.7.2 Code source	6
1.7.3 Schéma de propagation	6
1.8 Exercice 8 – Checked Exception	6
1.8.1 Objectif	7
1.8.2 Code source	7
1.8.3 Explication	7
1.9 Exercice 9 – Exception Personnalisée Vérifiée	7
1.9.1 Objectif	7
1.9.2 Code source	7
1.10 Exercice 10 – Comparaison Checked / Unchecked	8

1.10.1	Objectif	8
1.10.2	Code source	8
1.10.3	Comparaison	9
2	Exceptions Personnalisées	10
2.1	Exercice 1 – Compte Bancaire	10
2.1.1	Objectif	10
2.1.2	Code source	10
2.1.3	Explication	11
2.2	Exercice 2 – Montant Invalide	11
2.2.1	Objectif	11
2.2.2	Code source	11
2.2.3	Explication	11
2.3	Exercice 3 – Double Validation	12
2.3.1	Objectif	12
2.3.2	Code source	12
2.4	Exercice 4 – Inscription Utilisateur	12
2.4.1	Objectif	12
2.4.2	Code source	12
2.5	Exercice 5 – Authentification	13
2.5.1	Objectif	13
2.5.2	Code source	13
2.6	Exercice 6 – Gestion de Stock	14
2.6.1	Objectif	14
2.6.2	Code source	14
2.7	Exercice 7 – Téléchargement	14
2.7.1	Objectif	14
2.7.2	Code source	14
2.8	Exercice 8 – Validation de Formulaire	15
2.8.1	Objectif	15
2.8.2	Code source	15
2.9	Exercice 9 – Paiement	15
2.9.1	Objectif	15
2.9.2	Code source	15
2.10	Exercice 10 – Conception : Quand Utiliser Chaque Type?	16
2.10.1	Objectif	16
2.10.2	Synthèse	16
3	Généricité	17
3.1	Exercice 1 – Classe Générique Boite<T>	17
3.1.1	Objectif	17
3.1.2	Code source	17
3.1.3	Explication	18
3.2	Exercice 2 – Classe Paire<T, U>	18
3.2.1	Objectif	18
3.2.2	Code source	18
3.3	Exercice 3 – Interface Générique Calcul<T>	18
3.3.1	Objectif	18
3.3.2	Code source	18

3.4	Exercice 4 – Interface <code>Comparable<T></code>	19
3.4.1	Objectif	19
3.4.2	Code source	19
3.4.3	Explication	19
3.5	Exercice 5 – Deuxième Implémentation de <code>Comparable<T></code>	20
3.5.1	Objectif	20
3.5.2	Code source	20
3.6	Exercice 6 – Méthodes Génériques Statiques	20
3.6.1	Objectif	20
3.6.2	Code source	20
3.7	Exercice 7 – Borne Supérieure <code><T extends Number></code>	21
3.7.1	Objectif	21
3.7.2	Code source	21
3.7.3	Explication	21
3.8	Exercice 8 – Héritage Générique (Animal / Chien)	22
3.8.1	Objectif	22
3.8.2	Code source	22
3.9	Exercice 9 – Héritage Paramétré (Vehicule / Voiture)	22
3.9.1	Objectif	22
3.9.2	Code source	22
3.10	Exercice 10 – Repository Générique	23
3.10.1	Objectif	23
3.10.2	Code source	23
3.11	Exercice 11 – Wildcard <code><?></code>	23
3.11.1	Objectif	24
3.11.2	Code source	24
3.11.3	Explication	24
3.12	Exercice 12 – Wildcard Borné <code><? extends Number></code>	24
3.12.1	Objectif	24
3.12.2	Code source	24
3.12.3	Comparaison des Wildcards	25
	Conclusion	26

Chapitre 1

Exceptions Système

1.1 | Exercice 1 – Division Sécurisée

1.1.1 Objectif

Calculer la division de deux entiers saisis par l'utilisateur en gérant la division par zéro et la saisie invalide.

1.1.2 Code source

```
1 import java.util.Scanner;
2 import java.util.InputMismatchException;
3
4 public class exo1 {
5
6     public exo1() {}
7
8     public void calculer(exo1 e, int a, int b) {
9         try {
10             if (b == 0) {
11                 throw new ArithmeticException();
12             }
13             System.out.println(a / b);
14         } catch (ArithmeticException ex) {
15             System.out.println("Erreur : Division par zero.");
16         } catch (Exception ex) {
17             System.out.println("Erreur inattendue.");
18         }
19     }
20
21     public static void main(String[] args) {
22         Scanner sc = new Scanner(System.in);
23         try {
24             System.out.print("Enter a : ");
25             int a = sc.nextInt();
26             System.out.print("Enter b : ");
27             int b = sc.nextInt();
```

```
28         exo1 d = new exo1();
29         d.calculer(d, a, b);
30     } catch (InputMismatchException ex) {
31         System.out.println("Erreur : Saisie invalide, entrez
32         des entiers.");
33     }
34     sc.close();
35 }
```

Listing 1.1 – exo1.java – Division sécurisée

1.1.3 Explication

Le bloc `try/catch` dans la méthode `calculer()` intercepte l'`ArithmeticException` levée explicitement via `throw` lorsque le diviseur est nul. Dans le `main`, un second `try/catch` capture l'`InputMismatchException` levée par `Scanner` en cas de saisie non entière.

Le mot-clé `throw` permet de lever manuellement une exception depuis n'importe quel point du code.

1.2 | Exercice 2 – NullPointerException

1.2.1 Objectif

Afficher la longueur d'une chaîne en gérant le cas où elle est `null`, d'abord sans `try/catch` puis avec.

1.2.2 Code source

```
1 public class exo2 {
2
3     public void longueurSansCatch(exo2 e, String s) {
4         if (s == null) {
5             System.out.println("Null");
6         } else {
7             System.out.println(s.length());
8         }
9     }
10
11     public void longueurAvecCatch(exo2 e, String s) {
12         try {
13             System.out.println(s.length());
14         } catch (NullPointerException ex) {
15             System.out.println("Null");
16         }
17     }
18 }
```

Listing 1.2 – exo2.java – NullPointerException

1.2.3 Explication

La version sans `try/catch` utilise une condition `if` pour tester la nullité avant d'appeler la méthode. La version avec `try/catch` laisse la `NullPointerException` se lever et la capture dans le bloc `catch`.

1.3 | Exercice 3 – Tableau Sécurisé

1.3.1 Objectif

Permettre à l'utilisateur d'accéder à un index d'un tableau de taille 5 en proposant deux approches.

1.3.2 Code source

```
1 public class exo3 {
2
3     public void versionIf(exo3 e, int[] tab, int index) {
4         if (index >= 0 && index < 5) {
5             System.out.println(tab[index]);
6         } else {
7             System.out.println("Hors limites");
8         }
9     }
10
11    public void versionCatch(exo3 e, int[] tab, int index) {
12        try {
13            System.out.println(tab[index]);
14        } catch (ArrayIndexOutOfBoundsException ex) {
15            System.out.println("Hors limites");
16        }
17    }
18 }
```

Listing 1.3 – exo3.java – Accès sécurisé au tableau

1.3.3 Comparaison des deux approches

Critère	Version if	Version try/catch
Lisibilité	Explicite	Nécessite de connaître l'exception
Performance	Légèrement plus rapide	Surcoût si exception levée
Usage recommandé	Vérification préventive	Cas imprévisibles

Table 1.1 – Comparaison entre vérification par if et try/catch

1.4 | Exercice 4 – Conversion d'Entier

1.4.1 Objectif

Convertir une chaîne saisie en entier et afficher un message clair en cas d'échec.

1.4.2 Code source

```

1 public class exo4 {
2
3     public void convertir(exo4 e, String s) {
4         try {
5             int val = Integer.parseInt(s);
6             System.out.println(val);
7         } catch (NumberFormatException ex) {
8             System.out.println("Erreur : La chaine saisie ne peut
9                 pas etre convertie en entier.");
10        }
11    }
12 }
```

Listing 1.4 – exo4.java – NumberFormatException

1.4.3 Explication

`Integer.parseInt()` lève une `NumberFormatException` si la chaîne ne représente pas un entier valide. Ce type d'exception est une *unchecked exception* (hérite de `RuntimeException`).

1.5 | Exercice 5 – Choix d'Exception

1.5.1 Objectif

Créer une méthode `racineCarree(int x)` qui lève une exception si $x < 0$.

1.5.2 Code source

```
1 public class exo5 {  
2  
3     public int racineCarree(exo5 e, int x) {  
4         if (x < 0) {  
5             throw new IllegalArgumentException();  
6         }  
7         return (int) Math.sqrt(x);  
8     }  
9 }
```

Listing 1.5 – exo5.java – IllegalArgumentException

1.5.3 Justification du choix

`IllegalArgumentException` est choisie car l'erreur provient d'un **argument invalide** passé par le développeur. C'est une *unchecked exception* qui ne nécessite pas de `throws` dans la signature, ce qui est adapté à ce type de précondition logique.

1.6 | Exercice 6 – IllegalStateException

1.6.1 Objectif

Simuler une machine avec un état ON/OFF ; lever une exception si on tente de la démarrer alors qu'elle est déjà allumée.

1.6.2 Code source

```
1 public class exo6 {  
2  
3     boolean estAllumee;  
4  
5     public exo6() {  
6         this.estAllumee = false;  
7     }  
8  
9     public void demarrer(exo6 e) {  
10        if (e.estAllumee) {  
11            throw new IllegalStateException();  
12        }  
13        e.estAllumee = true;  
14        System.out.println("Machine ON");  
15    }  
16 }
```

Listing 1.6 – exo6.java – IllegalStateException

1.6.3 Explication

`IllegalStateException` est l'exception standard Java à utiliser lorsqu'une méthode est appelée à un moment où l'état de l'objet ne le permet pas.

1.7 | Exercice 7 – Propagation d'Exception

1.7.1 Objectif

Illustrer la propagation d'exceptions à travers une chaîne d'appels $A \rightarrow B \rightarrow C$.

1.7.2 Code source

```
1 public class exo7 {
2
3     public void methodeC(exo7 e) throws Exception {
4         throw new Exception();
5     }
6
7     public void methodeB(exo7 e) throws Exception {
8         e.methodeC(e);
9     }
10
11    public void methodeA(exo7 e) {
12        try {
13            e.methodeB(e);
14        } catch (Exception ex) {
15            System.out.println("Erreur interceptee par A");
16        }
17    }
18 }
```

Listing 1.7 – exo7.java – Propagation

1.7.3 Schéma de propagation

C lance l'exception → B la propage (throws) → A la capture (catch)

1.8 | Exercice 8 – Checked Exception

1.8.1 Objectif

Créer une méthode `verifierAge(int age) throws Exception` qui lève une exception si l'âge est inférieur à 18.

1.8.2 Code source

```
1 public class exo8 {
2
3     public void verifierAge(exo8 e, int age) throws Exception {
4         if (age < 18) {
5             throw new Exception();
6         }
7     }
8
9     public static void main(String[] args) {
10         exo8 e = new exo8();
11         try {
12             e.verifierAge(e, 15);
13             System.out.println("Age valide.");
14         } catch (Exception ex) {
15             System.out.println("Erreur : Age inferieur a 18.");
16         }
17     }
18 }
```

Listing 1.8 – exo8.java – Checked Exception

1.8.3 Explication

En utilisant `throws Exception`, le compilateur **oblige** l'appelant à gérer l'exception. C'est le comportement des *checked exceptions*.

1.9 | Exercice 9 – Exception Personnalisée Vérifiée

1.9.1 Objectif

Créer une `AgeException` personnalisée et l'utiliser pour vérifier un âge.

1.9.2 Code source

```
1 class AgeException extends Exception {}
2
3 public class exo9 {
4
5     public void verifierAge(exo9 e, int age) throws AgeException
6     {
7         if (age < 18) {
8             throw new AgeException();
9         }
10    }
```

```
8      }
9  }
10
11  public static void main(String[] args) {
12      exo9 e = new exo9();
13      try {
14          e.verifierAge(e, 15);
15          System.out.println("Age valide.");
16      } catch (AgeException ex) {
17          System.out.println("Erreur : Age inferieur a 18.");
18      }
19  }
20 }
```

Listing 1.9 – exo9.java – AgeException personnalisée

1.10 | Exercice 10 – Comparaison Checked / Unchecked

1.10.1 Objectif

Comparer les deux approches : Exception (checked) et RuntimeException (unchecked).

1.10.2 Code source

```
1  public class exo10 {
2
3      public void verifierChecked(exo10 e, int age) throws
4          Exception {
5          if (age < 18) {
6              throw new Exception();
7          }
8      }
9
10     public void verifierUnchecked(exo10 e, int age) {
11         if (age < 18) {
12             throw new RuntimeException();
13         }
14     }
```

Listing 1.10 – exo10.java – Comparaison Checked vs Unchecked

1.10.3 Comparaison

Critère	Exception (Checked)	RuntimeException (Unchecked)
Vérification	À la compilation	Seulement à l'exécution
Signature	Oblige <code>throws</code>	Aucun <code>throws</code> requis
Usage	Erreurs récupérables externes	Bugs logiques du développeur

Table 1.2 – Comparaison entre `Exception` et `RuntimeException`

Exceptions Personnalisées

2.1 | Exercice 1 – Compte Bancaire

2.1.1 Objectif

Créer une classe `CompteBancaire` avec `retirer()` et `verser()`, et lever une `SoldeInsuffisantException` si le montant dépasse le solde.

2.1.2 Code source

```
1 class SoldeInsuffisantException extends Exception {}
2
3 public class exo1_2 {
4
5     String code;
6     double solde;
7
8     public exo1_2(String code, double solde) {
9         this.code = code;
10        this.solde = solde;
11    }
12
13    void verser(exo1_2 e, double montant) {
14        e.solde += montant;
15        System.out.println("Nouveau solde : " + e.solde);
16    }
17
18    void retirer(exo1_2 e, double montant) throws
19        SoldeInsuffisantException {
20        if (montant > e.solde) {
21            throw new SoldeInsuffisantException();
22        }
23        e.solde -= montant;
24        System.out.println("Nouveau solde : " + e.solde);
25    }
26 }
```

Listing 2.1 – exo1_2.java – SoldeInsuffisantException

2.1.3 Explication

`SoldeInsuffisantException` hérite de `Exception` (checked). Le mot-clé `throws` dans la signature de `retirer()` oblige l'appelant à gérer cette situation.

2.2 | Exercice 2 – Montant Invalide

2.2.1 Objectif

Créer une `MontantInvalideException` levée lorsque le montant est ≤ 0 .

2.2.2 Code source

```
1 class MontantInvalideException extends RuntimeException {}
2
3 public class exo2_2 {
4
5     String code;
6     double solde;
7
8     public exo2_2(String code, double solde) {
9         this.code = code;
10        this.solde = solde;
11    }
12
13    void verser(exo2_2 e, double montant) {
14        if (montant <= 0) {
15            throw new MontantInvalideException();
16        }
17        e.solde += montant;
18        System.out.println("Nouveau solde : " + e.solde);
19    }
20 }
```

Listing 2.2 – exo2_2.java – MontantInvalideException

2.2.3 Explication

`MontantInvalideException` hérite de `RuntimeException` (unchecked), car un montant invalide est une erreur de logique qui ne devrait jamais se produire en production correcte.

2.3 | Exercice 3 – Double Validation

2.3.1 Objectif

Modifier `retirer()` pour gérer à la fois `MontantInvalideException` et `SoldeInsuffisantException`.

2.3.2 Code source

```
1 class SoldeInsuffisantException3 extends Exception {}
2 class MontantInvalideException3 extends RuntimeException {}
3
4 public class exo3_2 {
5
6     String code;
7     double solde;
8
9     public exo3_2(String code, double solde) {
10         this.code = code;
11         this.solde = solde;
12     }
13
14     void retirer(exo3_2 e, double montant) throws
        SoldeInsuffisantException3 {
15         if (montant <= 0) {
16             throw new MontantInvalideException3();
17         }
18         if (montant > e.solde) {
19             throw new SoldeInsuffisantException3();
20         }
21         e.solde -= montant;
22         System.out.println("Nouveau solde : " + e.solde);
23     }
24 }
```

Listing 2.3 – exo3_2.java – Double validation

2.4 | Exercice 4 – Inscription Utilisateur

2.4.1 Objectif

Valider l'email et l'âge lors d'une inscription via deux exceptions personnalisées.

2.4.2 Code source

```
1 class EmailInvalideException4 extends Exception {}
2 class AgeInvalideException4 extends Exception {}
3
4 public class exo4_2 {
```

```
5 void inscrire(exo4_2 e, String email, int age)
6     throws EmailInvalideException4, AgeInvalideException4
7     {
8         if (!email.contains("@")) {
9             throw new EmailInvalideException4();
10        }
11        if (age < 18) {
12            throw new AgeInvalideException4();
13        }
14        System.out.println("Inscription reussie");
15    }
16 }
```

Listing 2.4 – exo4_2.java – EmailInvalideException & AgeInvalideException

2.5 | Exercice 5 – Authentication

2.5.1 Objectif

Lever une `AuthenticationException` avec le message « Identifiants incorrects » en cas de mauvais login.

2.5.2 Code source

```
1 class AuthenticationException5 extends Exception {}
2
3 public class exo5_2 {
4
5     void login(exo5_2 e, String username, String password)
6         throws AuthenticationException5 {
7         if (!username.equals("admin") || !password.equals("1234"))
8             {
9                 throw new AuthenticationException5();
10            }
11        System.out.println("Connexion reussie");
12    }
13
14    public static void main(String[] args) {
15        exo5_2 e = new exo5_2();
16        try {
17            e.login(e, "user", "wrong");
18        } catch (AuthenticationException5 ex) {
19            System.out.println("Identifiants incorrects");
20        }
21    }
22 }
```

Listing 2.5 – exo5_2.java – AuthenticationException

2.6 | Exercice 6 – Gestion de Stock

2.6.1 Objectif

Lever une `StockInsuffisantException` si la quantité demandée dépasse le stock disponible.

2.6.2 Code source

```
1 class StockInsuffisantException6 extends Exception {}
2
3 public class exo6_2 {
4     int stock;
5
6     public exo6_2(int stock) {
7         this.stock = stock;
8     }
9
10    void retirerDuStock(exo6_2 p, int quantite)
11        throws StockInsuffisantException6 {
12        if (quantite > p.stock) {
13            throw new StockInsuffisantException6();
14        }
15        p.stock -= quantite;
16        System.out.println("Stock restant : " + p.stock);
17    }
18 }
```

Listing 2.6 – exo6_2.java – StockInsuffisantException

2.7 | Exercice 7 – Téléchargement

2.7.1 Objectif

Lever une `QuotaDepasseException` si la taille d'un fichier dépasse la limite autorisée.

2.7.2 Code source

```
1 class QuotaDepasseException7 extends Exception {}
2
3 public class exo7_2 {
4     double limite = 500.0;
5
6     void telechargerFichier(exo7_2 e, double taille)
7         throws QuotaDepasseException7 {
8         if (taille > e.limite) {
9             throw new QuotaDepasseException7();
10        }
11        System.out.println("Téléchargement fini");
12    }
13 }
```

```
12     }  
13 }
```

Listing 2.7 – exo7_2.java – QuotaDepasseException

2.8 | Exercice 8 – Validation de Formulaire

2.8.1 Objectif

Lever une `ChampObligatoireException` si un champ du formulaire est vide ou nul.

2.8.2 Code source

```
1 class ChampObligatoireException8 extends Exception {}  
2  
3 public class exo8_2 {  
4  
5     void validerFormulaire(exo8_2 e, String nom, String email)  
6         throws ChampObligatoireException8 {  
7         if (nom == null || nom.isEmpty() || email == null ||  
8             email.isEmpty()) {  
9             throw new ChampObligatoireException8();  
10        }  
11        System.out.println("Formulaire valide");  
12    }  
13 }
```

Listing 2.8 – exo8_2.java – ChampObligatoireException

2.9 | Exercice 9 – Paiement

2.9.1 Objectif

Gérer deux exceptions lors d'un paiement : dépassement de plafond et carte expirée.

2.9.2 Code source

```
1 class PaiementRefuseException9 extends Exception {}  
2 class CarteExpireeException9 extends Exception {}  
3  
4 public class exo9_2 {  
5     double plafond = 2000.0;  
6  
7     void payer(exo9_2 e, double montant, boolean expiree)  
8         throws PaiementRefuseException9,  
9             CarteExpireeException9 {  
10        if (expiree) {  
11            throw new CarteExpireeException9();  
12        }  
13    }  
14 }
```

```

11     }
12     if (montant > e.plafond) {
13         throw new PaiementRefuseException9();
14     }
15     System.out.println("Paiement effectue");
16 }
17 }

```

Listing 2.9 – exo9_2.java – PaiementRefuseException & CarteExpireeException

2.10 | Exercice 10 – Conception : Quand Utiliser Chaque Type ?

2.10.1 Objectif

Répondre aux questions de conception sur le choix du type d'exception.

2.10.2 Synthèse

Question	Réponse
Quand utiliser <i>checked</i> ?	Pour les erreurs recupérables et externes : fichier introuvable, connexion réseau, base de données. Le compilateur force la gestion.
Quand utiliser <i>unchecked</i> ?	Pour les bugs logiques du développeur : index hors limites, argument nul, montant invalide. Pas de throws requis.
Pourquoi créer une exception personnalisée ?	Pour donner un sens métier précis à l'erreur, améliorer la lisibilité du code et faciliter la maintenance.

Table 2.1 – Synthèse sur la conception des exceptions

Généricité

3.1 | Exercice 1 – Classe Générique Boite<T>

3.1.1 Objectif

Créer une classe générique `Boite<T>` avec les méthodes `setContenu` et `getContenu`, testée avec `String` et `Integer`.

3.1.2 Code source

```
1 public class exo1_3<T> {
2     T contenu;
3
4     void setContenu(exo1_3<T> e, T contenu) {
5         e.contenu = contenu;
6     }
7
8     T getContenu(exo1_3<T> e) {
9         return e.contenu;
10    }
11
12    public static void main(String[] args) {
13        exo1_3<String> boiteS = new exo1_3<>();
14        boiteS.setContenu(boiteS, "Java");
15        System.out.println("Contenu String : " + boiteS.
16                             getContenu(boiteS));
17
18        exo1_3<Integer> boiteI = new exo1_3<>();
19        boiteI.setContenu(boiteI, 42);
20        System.out.println("Contenu Integer : " + boiteI.
21                             getContenu(boiteI));
22    }
23 }
```

Listing 3.1 – exo1_3.java – Boite générique

3.1.3 Explication

Le paramètre de type `T` est résolu à la compilation selon le type utilisé lors de l'instanciation. Cela garantit la **sûreté de type** (*type-safety*) sans duplication de code.

3.2 | Exercice 2 – Classe Paire<T, U>

3.2.1 Objectif

Créer une classe générique à deux paramètres de type avec une méthode `afficherPaire()`.

3.2.2 Code source

```
1 public class exo2_3<T, U> {
2     T premier;
3     U second;
4
5     public exo2_3(T premier, U second) {
6         this.premier = premier;
7         this.second = second;
8     }
9
10    void afficherPaire(exo2_3<T, U> p) {
11        System.out.println("Paire -> Premier: " + p.premier
12                           + ", Second: " + p.second);
13    }
14
15    public static void main(String[] args) {
16        exo2_3<String, Integer> paire = new exo2_3<>("Java",
17                                                       2026);
18        paire.afficherPaire(paire);
19    }
20 }
```

Listing 3.2 – exo2_3.java – Paire générique

3.3 | Exercice 3 – Interface Générique Calcul<T>

3.3.1 Objectif

Définir une interface générique `Calcul<T>` et l'implémenter pour `Integer` et `Double`.

3.3.2 Code source

```
1 interface Calcul<T> {
2     T addition(T a, T b);
3 }
```

```
4
5 class CalculInt implements Calcul<Integer> {
6     public Integer addition(Integer a, Integer b) {
7         return a + b;
8     }
9 }
10
11 class CalculDouble implements Calcul<Double> {
12     public Double addition(Double a, Double b) {
13         return a + b;
14     }
15 }
```

Listing 3.3 – exo3_3.java – Interface Calcul générique

3.4 | Exercice 4 – Interface Comparateur<T>

3.4.1 Objectif

Définir une interface `Comparateur<T>` et l'implémenter pour `Integer` (par valeur) et `String` (par longueur).

3.4.2 Code source

```
1 interface Comparateur4<T> {
2     int comparer(T a, T b);
3 }
4
5 class ComparateurInt4 implements Comparateur4<Integer> {
6     public int comparer(Integer a, Integer b) {
7         return a.compareTo(b);
8     }
9 }
10
11 class ComparateurString4 implements Comparateur4<String> {
12     public int comparer(String a, String b) {
13         return Integer.compare(a.length(), b.length());
14     }
15 }
```

Listing 3.4 – exo4_3.java – Interface Comparateur

3.4.3 Explication

La valeur retournée par `comparer()` suit la convention Java : négatif si $a < b$, zéro si $a = b$, positif si $a > b$.

3.5 | Exercice 5 – Deuxième Implémentation de Comparateur<T>

3.5.1 Objectif

Réimplémenter l'interface `Comparateur<T>` dans un fichier indépendant pour consolider la compréhension.

3.5.2 Code source

```
1 interface Comparateur5<T> {
2     int comparer(T a, T b);
3 }
4
5 class ComparateurInt5 implements Comparateur5<Integer> {
6     public int comparer(Integer a, Integer b) {
7         return a.compareTo(b);
8     }
9 }
10
11 class ComparateurString5 implements Comparateur5<String> {
12     public int comparer(String a, String b) {
13         return Integer.compare(a.length(), b.length());
14     }
15 }
```

Listing 3.5 – exo5_3.java – Comparateur5

3.6 | Exercice 6 – Méthodes Génériques Statiques

3.6.1 Objectif

Créer deux méthodes statiques génériques : `afficherTableau(T[])` et `getPremier(T[])`.

3.6.2 Code source

```
1 public class exo6_3 {
2
3     public static <T> void afficherTableau(T[] tableau) {
4         for (T element : tableau) {
5             System.out.print(element + " ");
6         }
7         System.out.println();
8     }
9
10    public static <T> T getPremier(T[] tableau) {
11        if (tableau.length > 0) {
```

```
12         return tableau[0];
13     }
14     return null;
15 }
16
17 public static void main(String[] args) {
18     Integer[] nombres = {10, 20, 30, 40, 50};
19     String[] mots = {"Java", "Data", "ENSAH"};
20
21     afficherTableau(nombres);
22     System.out.println("Premier : " + getPremier(nombres));
23     afficherTableau(mots);
24     System.out.println("Premier : " + getPremier(mots));
25 }
26 }
```

Listing 3.6 – exo6_3.java – Méthodes génériques statiques

3.7 | Exercice 7 – Borne Supérieure <T extends Number>

3.7.1 Objectif

Créer une méthode générique bornée qui calcule la somme de deux nombres de n'importe quel type numérique.

3.7.2 Code source

```
1 public class exo7_3 {
2
3     public static <T extends Number> double somme(T a, T b) {
4         return a.doubleValue() + b.doubleValue();
5     }
6
7     public static void main(String[] args) {
8         System.out.println("Somme : " + somme(5, 3.14));
9     }
10 }
```

Listing 3.7 – exo7_3.java – Borne supérieure

3.7.3 Explication

La borne <T extends Number> restreint le paramètre de type aux sous-classes de Number (Integer, Double, Float...), permettant d'appeler doubleValue() en toute sécurité.

3.8 | Exercice 8 – Héritage Générique (Animal / Chien)

3.8.1 Objectif

Créer une classe générique `Animal<T>` et une sous-classe `Chien` qui fixe le type à `String`.

3.8.2 Code source

```
1 class Animal<T> {
2     T nom;
3 }
4
5 class Chien extends Animal<String> {
6     public Chien(String nom) {
7         this.nom = nom;
8     }
9 }
10
11 public class exo8_3 {
12     public static void main(String[] args) {
13         Chien c = new Chien("Rex");
14         System.out.println("Nom du chien : " + c.nom);
15     }
16 }
```

Listing 3.8 – exo8_3.java – Héritage générique

3.9 | Exercice 9 – Héritage Paramétré (Vehicule / Voiture)

3.9.1 Objectif

Créer une classe générique `Vehicule<T>` et une sous-classe `Voiture<T>` qui hérite du paramètre de type.

3.9.2 Code source

```
1 class Vehicule<T> {
2     T vitesse;
3 }
4
5 class Voiture<T> extends Vehicule<T> {
6     public Voiture(T vitesse) {
```

```
7         this.vitesse = vitesse;
8     }
9 }
10
11 public class exo9_3 {
12     public static void main(String[] args) {
13         Voiture<Integer> v = new Voiture<>(120);
14         System.out.println("Vitesse : " + v.vitesse);
15     }
16 }
```

Listing 3.9 – exo9_3.java – Vehicule et Voiture génériques

3.10 | Exercice 10 – Repository Générique

3.10.1 Objectif

Créer un Repository<T> générique et une classe UserRepository qui le spécialise pour les objets User.

3.10.2 Code source

```
1 class User {
2     String nom;
3     public User(String nom) { this.nom = nom; }
4     public String toString() { return "User: " + nom; }
5 }
6
7 class Repository<T> {
8     void save(T obj) {
9         System.out.println("Objet sauvegarde : " + obj.toString()
10             );
11     }
12 }
13
14 class UserRepository extends Repository<User> {}
15
16 public class exo10_3 {
17     public static void main(String[] args) {
18         User u = new User("Yazid");
19         UserRepository repo = new UserRepository();
20         repo.save(u);
21     }
22 }
```

Listing 3.10 – exo10_3.java – Repository générique

3.11 | Exercice 11 – Wildcard <?>

3.11.1 Objectif

Créer une méthode `afficherListe(List<?> liste)` acceptant n'importe quel type de liste.

3.11.2 Code source

```
1 import java.util.List;
2 import java.util.ArrayList;
3
4 public class exo11_3 {
5
6     void afficherListe(List<?> liste) {
7         for (Object obj : liste) {
8             System.out.print(obj + " ");
9         }
10        System.out.println();
11    }
12
13    public static void main(String[] args) {
14        List<String> noms = new ArrayList<>();
15        noms.add("Yazid");
16        noms.add("ENSAH");
17
18        exo11_3 e = new exo11_3();
19        e.afficherListe(noms);
20    }
21 }
```

Listing 3.11 – exo11_3.java – Wildcard ?

3.11.3 Explication

Le wildcard `<?>` signifie « n'importe quel type ». Il permet d'écrire une méthode générique sans spécifier de paramètre de type, mais en contrepartie on ne peut **pas** ajouter d'éléments à la liste (sauf `null`).

3.12 | Exercice 12 – Wildcard Borné `<? extends Number>`

3.12.1 Objectif

Créer une méthode `afficherNombres(List<? extends Number>)` testée avec `List<Integer>` et `List<Double>`.

3.12.2 Code source

```

1 import java.util.List;
2 import java.util.ArrayList;
3
4 public class exo12_3 {
5
6     void afficherNombres(List<? extends Number> liste) {
7         for (Number n : liste) {
8             System.out.print(n + " ");
9         }
10        System.out.println();
11    }
12
13    public static void main(String[] args) {
14        List<Integer> entiers = new ArrayList<>();
15        entiers.add(10);
16        entiers.add(20);
17
18        List<Double> reels = new ArrayList<>();
19        reels.add(15.5);
20        reels.add(25.5);
21
22        exo12_3 e = new exo12_3();
23        e.afficherNombres(entiers);
24        e.afficherNombres(reels);
25    }
26 }

```

Listing 3.12 – exo12_3.java – Wildcard borné

3.12.3 Comparaison des Wildcards

Wildcard	Signification	Usage typique
<?>	N'importe quel type	Lecture seule, type inconnu
<? extends T>	T ou ses sous-types	Lecture d'une collection de T
<? super T>	T ou ses super-types	Écriture dans une collection

Table 3.1 – Comparaison des wildcards en Java

Conclusion

Ce travail pratique nous a permis de maîtriser deux piliers fondamentaux de la programmation orientée objet en Java : la **gestion des exceptions** et la **généricité**.

La première partie a mis en évidence la différence entre exceptions vérifiées (*checked*) et non vérifiées (*unchecked*), ainsi que l'importance de créer des hiérarchies d'exceptions métier claires et expressives. La propagation d'exceptions à travers la pile d'appels et l'usage du mot-clé `throw` ont été mis en pratique de manière concrète.

La deuxième partie a montré la puissance des exceptions personnalisées pour rendre le code plus lisible, robuste et maintenable, en donnant un sens métier précis aux erreurs détectées à l'exécution.

Enfin, la troisième partie a illustré l'intérêt de la générique pour écrire du code réutilisable, type-safe, applicable à des structures de données, des interfaces et des classes génériques avec bornes et wildcards.

Yazid TAHIRI ALAOUI
ENSAH – 2025 – 2026